How to use Geospatial Web Lab

Setup, workflows, and recovery | 29 September 2026

Setup, operation, and demonstration guide for the five learning applications. All bundled operational data is synthetic. Use README.md for the project overview and PROJECT_SUMMARY.md for portfolio context. Verification evidence is recorded in VERIFICATION.md, dated September 29, 2026.

1. Prerequisites

Keep the five project folders, `shared`, and the root npm workspace together. The frontends import shared components and cannot be built from isolated project folders.

The validated local arrangement uses Node 24 on Windows and Ubuntu 24.04 under WSL for Ruby 3.2.3, Rails 8.1.4, PostgreSQL 16/PostGIS 3.4, and Redis 7. Another Linux environment needs Ruby 3.2 or newer, Bundler 4.0.21 as recorded in the lockfiles, PostgreSQL with matching PostGIS packages, Redis, Node 24, and native build prerequisites including `build-essential`, `libpq-dev`, and `libyaml-dev`.

Install these prerequisites before running the project scripts. `bootstrap-wsl.sh` provisions local databases and starts services; it does not install Ruby, database packages, or application dependencies. Use a PostgreSQL client with the same major version as the server for database dumps.

Use a current WebGL-capable browser. ArcGIS basemaps and SDK assets need internet access. The Cesium corridor uses local assets and synthetic elevations without an ion token or terrain-service account.

2. First-time local setup

Install application dependencies

In PowerShell, change to the repository root and install the locked JavaScript dependencies:

```
cd C:\path\to\web-dev-projects
npm ci
```

In Ubuntu/WSL, change to the corresponding Linux path. Replace the example path below with your checkout. Install Bundler through your configured Ruby environment, then install gems for every backend:

```
cd /mnt/c/path/to/web-dev-projects
gem install bundler -v 4.0.21
project="$PWD"
for backend in 0*/backend; do
  (cd "$project/$backend" && bundle install)
done
```

Provision databases and seed demo records

From the repository root in Ubuntu/WSL:

```
sudo bash scripts/bootstrap-wsl.sh
bash scripts/seed-demo.sh
```

Bootstrap creates five development databases and five matching `_test` databases, with PostGIS installed by the PostgreSQL administrator. The local application role is `geolab`, with password `geolab`. It is not a superuser, but has database-creation permission for local setup and recovery checks. These are development credentials.

Seeding is an explicit operation that applies migrations and creates synthetic records and learning accounts. It can also validate or approve existing seeded datasets. Run it for initial demo preparation, not as a routine restart command. Startup itself never runs seeds.

Database names are `service_requests`, `data_quality_portal`, `fleet_monitor`, `parcel_scenarios`, and `infrastructure_inspections`. Backend configuration supports `DATABASE_URL`, `TEST_DATABASE_URL`, and `REDIS_URL` overrides. When running all five together, retain separate database and Redis queue configurations for each application.

3. Start and stop applications

Run all five

Keep two terminals open. In an Ubuntu/WSL terminal at the repository root:

```
bash scripts/start-backends.sh
```

In PowerShell at the repository root:

```
node scripts/dev-frontends.mjs
```

The backend launcher migrates databases, starts Rails and Sidekiq, and checks API health. If an API is already running, its migration step is skipped. Stop the existing supervisor before restarting after code or schema changes.

Application	Browser address	API port
Service requests	127.0.0.1:5171	3101
Data quality portal	127.0.0.1:5172	3102
Fleet monitor	127.0.0.1:5173	3103
Parcel scenarios	127.0.0.1:5174	3104
Infrastructure inspections	127.0.0.1:5175	3105

Use the same hostname consistently within a browser session. The local servers bind to loopback. Frontend requests proxy to their corresponding APIs and WebSocket endpoints.

Press Ctrl+C in each launcher terminal to stop its child processes. A child failure also stops the remaining children owned by that launcher. To stop managed APIs and workers from another Ubuntu terminal:

```
bash scripts/stop-backends.sh
```

PostgreSQL and Redis remain running. Stop those services separately only when no other local work needs them.

Run one application

After initial setup, start the chosen backend and worker in separate Ubuntu terminals. For service requests:

```
cd 01-service-requests/backend
bundle exec rails runner \
  'ActiveRecord::Base.connection_pool.schema_migration.create_table'
bundle exec rails db:migrate
bundle exec rails server -b 127.0.0.1 -p 3101
```

```
cd 01-service-requests/backend
bundle exec sidekiq -c 3
```

Then run `npm run dev:1` from the root in PowerShell. Substitute the project folder, API port, and `dev:2` through `dev:5` for another application. Stop these individual foreground processes with Ctrl+C. Do not run individual and combined launchers on the same ports.

4. Sign in and test roles

All seeded accounts use password `Learning123!`.

Account	Role	Use
`staff@example.test`	Staff	Operational management and approvals
`reporter@example.test`	Reporter	Restricted reporting, viewing, and owned records
`crew@example.test`	Staff	Second staff account in service requests only

Use separate browser profiles or a normal and private window for simultaneous users. Two ordinary tabs share the same application's cookie; signing in or out updates both tabs. Each application has its own cookie, so accounts are independent across applications.

Sessions expire after 24 hours. Logout revokes that login on the server. Account changes discard the previous account's drafts. Reauthenticate after an expired session or an upgrade that invalidates older cookies.

Failed sign-ins are limited: five for one email, or twenty from one address, within 15 minutes, after which the API answers "Too many failed sign-in attempts" until the window passes. Successful sign-ins are never counted and clear that email's failures.

5. Demonstrate each workflow

Service requests

1. Open port 5171 as the reporter. Choose **+ New request**, enter a title, category, description, latitude, and longitude, then **Create request**. For a nearby synthetic point, use latitude `33.045` and longitude `-96.995`.
2. Sign in as staff in a separate browser session. Locate the request on the map or register. Search, category, status, and **Distance filter** combine to narrow results.
3. Select a staff member and **Save changes** to assign the request. Advance through `assigned`, `in_progress`, and `resolved`. The reporter sees their own request update automatically. Reporters cannot assign work or access other reporters' requests.
4. For a concurrent-edit demonstration, open the same request in both staff accounts. Save one edit. The other draft requires **Load current values** before another save; it cannot overwrite newer values silently.

5. As staff, open **Imports & reports**. Choose **Download example**, then upload the downloaded GeoJSON. Review progress and **Review rejected rows** when present. Uploading the same content again reuses the existing owned import.
6. Choose **Generate CSV report**, wait for `completed`, then **Download CSV**. The report contains all requests, regardless of current map filters, and is downloadable only by its requesting staff account.

Imports accept 1 to 500 Point features under the 2 MB limit. Each needs a `title`; supported categories are `roads`, `lighting`, `drainage`, and `parks`, with `roads` as the default. Valid rows continue when other rows fail validation. Correct bad source rows and submit a new upload. **Retry import** is for a failed processing run, not for repairing invalid records. The map/register shows at most 500 matching requests while summary counts cover all matches.

Data quality portal

1. Open port 5172 as the reporter and select **Parks With Errors**. Its deliberate errors exercise missing attributes, reversed coordinates, an unclosed ring, and a self-intersecting polygon.
2. Choose **Upload GeoJSON**. Copy `02-data-quality-portal/backend/samples/parks-clean.geojson` to a new file and change one feature's `asset_id` value while preserving valid geometry. Select the edited file, set **Dataset name** and **Required attributes, comma separated**, then **Upload and validate**. Equivalent content and rules reuse the same uploader's existing dataset; changing only the dataset name does not create a new dataset.
3. Watch progress and inspect **Record review**. Filter accepted or rejected rows and select a row to inspect its source and findings. Only accepted features appear on the map.
4. In a separate staff session, select the dataset. Choose **Approve valid features**. For a mixed dataset, first acknowledge that rejected features will be excluded. A dataset with no accepted features cannot be approved.
5. Choose **Download approved GeoJSON**. The recorded version includes a feature count and SHA-256 digest. Corrected inputs require a new upload; the approved snapshot remains unchanged.

Files are limited to 2,000 features and 5 MB. Supported geometries are Point, MultiPoint, LineString, MultiLineString, Polygon, and MultiPolygon. Coordinates must contain exactly two finite numbers in longitude, latitude order in WGS84. Three-dimensional positions, GeometryCollection, empty/null geometry, invalid rings, and invalid topology are rejected. No automatic geometry repair or coordinate-system guessing occurs. Reporters see their own datasets; staff see all datasets and control approval. **Retry validation** appears for processing failures.

Fleet monitor

1. Open port 5173 as staff and choose **Start replay**. Ten simulated vehicles begin producing persisted positions. Select a vehicle from the list or map to inspect its route, trail, speed history, and zone membership.
2. Change **Replay speed** between 1x, 2x, and 4x. The multiplier processes more simulated frames per job; wall-clock cadence still depends on worker scheduling and host load.
3. Read **Zone event chronology**, optionally filtering to **Selected vehicle**. Entries and exits represent membership changes. Points on a polygon boundary count as inside.
4. Choose **Add zone**, provide a name and west/south/east/north bounds around part of the route, then **Create rectangular zone**. Membership is evaluated on subsequent accepted positions.
5. Choose **Pause** and verify positions stabilize. An observer signed in as the reporter can inspect results but cannot operate replay or modify zones.

Reset replay clears positions, trails, and transition history while retaining routes and geofences. Removing a zone also removes its associated history. Use these controls only when that loss is intended. History retains 360 points per vehicle and 500 events globally; the UI displays the latest 100 events. If an executing replay process is lost, pause and start replay to establish a new generation.

Parcel scenarios

1. Open port 5174. Both accounts can create scenarios; each sees only their own saved scenarios.

2. Choose a **District**, then select parcels on the map or table. **Select visible** selects every parcel in the currently filtered table, not only parcels inside the map viewport. Use **Clear selection** before changing study areas if prior selections should be removed.
3. Enter a **Scenario name**, **Floors**, **Unit area (sq ft)**, and **Building coverage**. For a first comparison, use 4 floors, 900 square feet per unit, and 40% coverage. Choose **Calculate … selected parcels**.
4. Wait for state `complete`. Save another named scenario with different assumptions. Select up to four completed rows to compare estimated units and open space.
5. Choose **Export** to download the calculation, original parcel geometry, and assumptions. Use **Retry** only for a failed scenario. Save a new scenario for changed assumptions; completed calculations are preserved. **Delete** removes the selected saved scenario.

Inputs allow 1 to 30 floors, 5% to 80% coverage, and unit areas from 400 to 3,000 square feet. Gross area equals site area multiplied by coverage and floors. Residential area uses an 80% efficiency assumption, and estimated units round down. Synthetic height limits produce warnings. These outputs are planning exercises, not permit, engineering, or financial determinations.

Infrastructure inspections

1. Open port 5175 and select an asset in **Asset register** or the Cesium scene. Use **Focus in 3D** and **View full corridor** to navigate the eight synthetic assets.
2. Change **Height display** between actual scale, 3x, and 5x. Compare base elevation, structure height, and top elevation. Exaggeration changes the drawing only; stored and exported measurements remain unchanged.
3. As the reporter, choose **Add observation**, select severity, enter 10 to 2,000 characters in **Observation notes**, and **Save observation**.
4. In a staff session, choose **Resolve observation**, supply resolution notes, then **Save status change**. Review **History**. Staff can **Reopen observation** with a reason. If another user changes an open draft, cancel and reopen the form before saving.
5. Choose **Generate profile**. Wait for `completed`, inspect the chart and **Profile samples**, then **Download profile GeoJSON**. A failed run exposes **Retry preprocessing**.

The profile snapshots source asset values and linearly interpolates synthetic base elevations, using PostGIS geography distances. Eight seeded assets produce 71 samples. The corridor surface is not a measured DEM or clearance assessment. Registers and observation forms remain usable if the 3D renderer cannot initialize.

6. Run verification

From the root in PowerShell:

```
npm test
npm run lint
node --test tests/runtime-scripts.node.mjs
node --test tests/shared-copies.node.mjs
npm run build
npx playwright install chromium
```

From the root in Ubuntu/WSL:

```
bash tests/runtime-shell.test.sh
bash scripts/test-backends.sh
```

With all five frontends, APIs, and workers running, execute `npm run test:e2e` from PowerShell. Backend suites use separate test databases. Browser tests use development services and leave synthetic records behind. Existing audit results are recorded evidence, not a substitute for rerunning checks after code changes.

7. Container setup

Image builds and runtime execution were not verified locally because the local Docker engine was unavailable; the verified local runtime is WSL. All five stacks built and passed the container smoke test on GitHub Actions on October 1, 2026. The container workflow builds and smoke-tests each stack on GitHub Actions with `scripts/container-smoke.sh`. You can run the same script locally, for example `bash scripts/container-smoke.sh 02-data-quality-portal`.

With Docker available, from the chosen project folder:

```
node ../scripts/configure-env.mjs
docker compose up --build -d
docker compose exec api bundle exec rails db:seed
```

The configuration script generates missing secrets for all five projects and preserves existing secrets. Seeding is optional and creates known demo credentials. The sign-in form prefills them unless `.env` sets `DEMO_ACCOUNTS=false` before the build; the form then starts empty and the bundle omits the demo password. Each Compose stack includes nginx, Rails, Sidekiq, PostGIS, and Redis. Migrations run before API and worker startup. `docker compose down` stops the stack while preserving volumes. Do not add `-v` when preserving data.

For an older database volume, back it up, generate the new application password, then run:

```
docker compose up -d db
docker compose exec db bash /docker-entrypoint-initdb.d/20-geolab.sh
docker compose up --build -d
```

The administrator initialization step grants the separate application role ownership of application tables while retaining administrator ownership of PostGIS. Do not delete the volume to upgrade it. Public hosting additionally requires HTTPS, real host/origin allowlists, secure cookies, consistent proxy/SSL configuration, and replacement of demo credentials, including a frontend built with `DEMO_ACCOUNTS=false`. The full-stack applications are not publicly hosted.

8. Preserve and recover data

From a project's folder, replace `DATABASE_NAME` with its database name:

```
docker compose exec -T db pg_dump -U geolab -d DATABASE_NAME -Fc `
  --exclude-table-data=public.spatial_ref_sys -f /tmp/backup.dump
docker compose cp db:/tmp/backup.dump ./backup.dump
```

Writing the binary dump inside the container and copying it avoids PowerShell redirection corruption. Restore into a separately created database using `pg_restore --no-owner --no-acl`, after an administrator provisions PostGIS. The extension supplies the excluded reference-table rows. Verify record counts and spatial queries before adopting a restored database; never use the working database as a restore experiment.

For the default local WSL arrangement, `sudo bash scripts/verify-backup.sh` checks service-request backup/restore in a temporary database, removes that temporary database, and leaves the dump in `.runtime/service-requests.dump`. It covers that application, not all five. Keep backups, credentials, `.env` files, and runtime logs out of a portfolio publication.

9. Troubleshooting

Symptom	Action
Browser cannot connect	Check both launcher terminals and the correct port. Stop duplicate launchers before restarting.
Missing gems or tables	Run backend `bundle install`, bootstrap, and migrations. Seeds are needed only for demo preparation.
Jobs remain queued	Check Redis and the project's worker. Inspect `.runtime/<project>-worker.log` for combined launches.
A run shows failed	A service-request import or export can show failed between automatic attempts. Active Job allows three total attempts, with retry delays starting at 3 and 18 seconds plus jitter and queue time. Validation allows three attempts only for `ActiveRecord::ConnectionNotEstablished`. Additional Sidekiq retries are disabled for imports, exports, validation, and scenarios; scenario calculations require manual retry. Profile errors recorded as failed require manual recovery, but errors escaping the failure handler can still trigger Sidekiq retries. If a run stays failed, fix the cause, then use **Retry import**, **Retry validation**, **Retry**, or **Retry preprocessing**; for a failed export, choose **Generate CSV report** again.
An import, validation or profile run stays processing	If its worker process was lost, the app marks it abandoned and shows **Resume import**, **Resume validation** or **Resume preprocessing**. A run a worker is still processing cannot be resumed.
Replay shows running but vehicles do not move	The worker running the replay may have stopped. Check the worker, then choose **Pause** and **Start replay**.
Sign-in says "Too many failed sign-in attempts"	Wait 15 minutes, or clear the counts in that backend with `bundle exec rails runner 'Rails.cache.clear'` (in Compose: `docker compose exec api bundle exec rails runner 'Rails.cache.clear'`).
API fails during startup	Inspect `.runtime/<project>-api.log`; check PostgreSQL, credentials, and migrations.
Live indicator is offline	Polling can recover persisted state. Check API, Redis, and WebSocket connectivity.
Blank map or scene	Check internet access for ArcGIS and WebGL support. Continue through the linked register where available.
Save reports stale data	Reload current values or reopen the edit form, then review before saving.
Login disappears in another tab	Tabs share that app's session. Use separate browser profiles for simultaneous roles.